

AI Intelligence Graph — Production Architecture

Goal: Build a fault-tolerant ingestion platform that can scale from the demonstrated 1K+ entity datasets to 500K+ records without application-code changes.

1. End-to-End Architecture

Layer	Design
Discovery	Source registry stores crawl targets, source type, priority, rate limits and freshness policy.
Crawlers	Async asyncio/aiohttp workers for feeds and APIs; Playwright Async workers for JS-heavy sources.
Queue	Kafka/SQS-style durable queue separates discovery, fetching, extraction and enrichment.
Raw storage	Object storage keeps immutable HTML, API responses and crawl metadata for replay and audit.
Extraction	Deterministic parsers first; LLM extraction only where unstructured text requires semantic normalization.
Validation	Pydantic schemas, URL validation, timestamp checks, freshness checks and record-level rejection.
Primary DB	PostgreSQL stores canonical entities, source lineage, timestamps, deduplication keys and relationships.
Graph / Vector	Neo4j for complex entity relationships; pgvector for semantic retrieval when needed.

2. Scaling to 500K+ Records

The crawler and extraction workers are stateless. A durable queue partitions work by source and record ID, allowing worker replicas to scale horizontally. Each job has an idempotency key and checkpoint so failures do not require restarting the entire crawl. Raw responses are written to object storage before downstream processing, allowing extraction workers to scale independently from network crawlers.

500K execution model: partition source URLs across workers → asynchronously fetch with bounded concurrency → persist raw payload → parse/LLM extract → validate → upsert canonical record. Increasing queue partitions and worker replicas increases throughput without changing application logic.

3. LLM Reliability: 413 and 429

Failure	Strategy
413 Payload Too Large	Split input by semantic paragraphs. Start with an 8K character chunk and halve the chunk size after a 413, down to a safe minimum.
429 Rate Limit	Retry up to 3 times using exponential backoff plus random jitter: $2^{\text{attempt}} + \text{random}(0,1)$. After retry exhaustion, fall through to the next provider.
Provider failure	Gemini Flash is primary, Groq is secondary, and DeepSeek is configured as an optional tertiary provider.
Invalid JSON	Reject the response and fall through/retry rather than accepting malformed or fabricated data.

4. Freshness, Deduplication and Distributed Crawling

Every source record receives a deterministic identity based on normalized source URL plus a content hash. A distributed deduplication store such as Redis prevents multiple workers from processing the same URL simultaneously. PostgreSQL enforces unique constraints on canonical identifiers and source URLs.

For news and jobs, the published timestamp is normalized to UTC. Records are accepted only when their publication time is within the configured 24-hour freshness window. When a source does not expose a strict timestamp, the system records the heuristic used rather than silently claiming precision.

Idempotency: source URL + normalized title/content hash → deterministic deduplication key → transactional upsert. This makes retries safe and prevents duplicate records across distributed crawler nodes.

5. Anti-Bot and JavaScript-Heavy Sources

Use direct HTTP requests for APIs/RSS and lightweight HTML sources. Escalate only blocked or JS-rendered sources to a bounded Playwright Async browser pool. Respect per-domain concurrency and delays, reuse browser contexts, and cache successful responses. Cloudflare/Datadome-protected sources should be handled through compliant browser automation or official feeds/APIs rather than attempting to bypass access controls.

6. Entity Resolution

Entity resolution is deterministic: Unicode normalization, lowercasing, punctuation cleanup and removal of legal suffixes are followed by lookup against a canonical seed dictionary. Examples such as OpenAI, OpenAI Inc. and Open AI resolve to OpenAI. Unmatched values remain unresolved instead of being guessed. Every mapping retains the raw value, canonical value, method and source URL for auditability.

7. Data Quality and Observability

- Pydantic validation before canonical persistence.
- Source URL retained for every record.
- Structured retry/error logging.
- Freshness checks for jobs and news.
- Live GitHub star collection for research repositories.
- Dead-letter queue for records that repeatedly fail extraction.
- Metrics: fetch latency, status codes, retry counts, extraction success rate, freshness rejection rate and duplicate rate.

8. Production Storage Choice

PostgreSQL is the system of record because the canonical schemas are relational, require constraints and need reliable upserts. Object storage preserves raw evidence and enables reprocessing. Neo4j is appropriate when traversing company ↔ product ↔ founder ↔ paper relationships becomes a core query pattern. pgvector can provide semantic search without introducing a separate vector service at smaller scale.

Current demonstrated pipeline: 1,000 startups, 1,202 products, 1,000 research papers, 43 fresh jobs and 15 fresh news records. The architecture separates ingestion, extraction, validation and storage so these workloads can scale independently.